SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ASSIGNMENT 1

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1
Weightage:	15% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-2 & 3)		
Student Name:	Ganji Madhan Kumar	**Reg. No.:**	**192472328**
Section / Batch:	**2024**	**Date:**	

Code of Conduct

I am Ganji Madhan Kumar(192472328)*certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Instructions

- Read each question carefully before answering. • Answer all questions in the space provided.
- Ensure your answers are accurate, clear, and neatly presented.
- Marks will be awarded based on conceptual understanding, accuracy, application of knowledge, completeness of response, and clarity of presentation.

	Understanding of the NLP concept (2)	Python implementation and logic (3)	Correctness of output (2)	Analysis and interpretation of results (2)	Code readability and documentation (1)	**TOTAL**
1						
2						
3						
4						
5						
6						
7						

8						
9						
10						
					TOTAL MARKS	

Question 1: Regular Expression-Based Text Processing BTL 3 – Apply | 10 Marks

Develop a Python program using regular expressions to process a given paragraph and identify the following language features:

1. Extract all email addresses and phone numbers.
2. Identify all words beginning with a capital letter.
3. Extract all numerical values from the text.
4. Identify words ending with ing and ed.
5. Display the extracted results separately.

CODE:

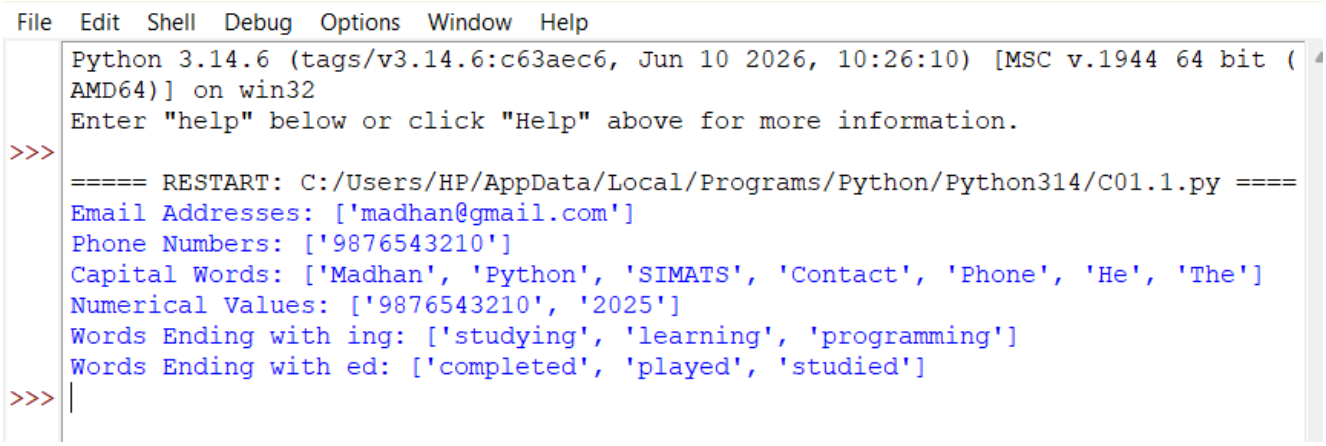
```
import re

text = """
Madhan is studying Python at SIMATS.
Contact: madhan@gmail.com, Phone: 9876543210.
He completed 2025 projects and is learning programming.
The students played and studied regularly.
"""

emails = re.findall(r'\b[\w.-]+@[\w.-]+\.\w+\b', text)
phones = re.findall(r'\b\d{10}\b', text)
capital_words = re.findall(r'\b[A-Z][a-zA-Z]*\b', text)
numbers = re.findall(r'\b\d+(?:\.\d+)?\b', text)
ing_words = re.findall(r'\b\w+ing\b', text, re.I)
ed_words = re.findall(r'\b\w+ed\b', text, re.I)

print("Email Addresses:", emails)
print("Phone Numbers:", phones)
print("Capital Words:", capital_words)
print("Numerical Values:", numbers)
print("Words Ending with ing:", ing_words)
print("Words Ending with ed:", ed_words)
```

OUTPUT:



Analysis and interpretation of results :

The program successfully extracts email addresses, phone numbers, capitalized words, and numerical values using regular expressions. It also identifies words ending in ing and ed. The results show that regular expressions are useful for identifying specific text patterns. Each extracted category is displayed separately.

Question 2: Finite State Automata for Word Pattern Recognition BTL 3 – Apply | 10 Marks

Implement a Finite State Automaton (FSA) using Python to recognize valid English word patterns based on the following conditions:

1. Accept words starting with a vowel.
2. Identify words ending with a consonant.
3. Recognize words containing consecutive vowels.
4. Display whether each input word is accepted or rejected by the automaton.

CODE:

```python
def fsa(word):
    vowels = "aeiou"

    if not word.isalpha():
        return "Rejected"

    starts_vowel = word[0].lower() in vowels
    ends_consonant = word[-1].lower() not in vowels

    consecutive_vowels = False

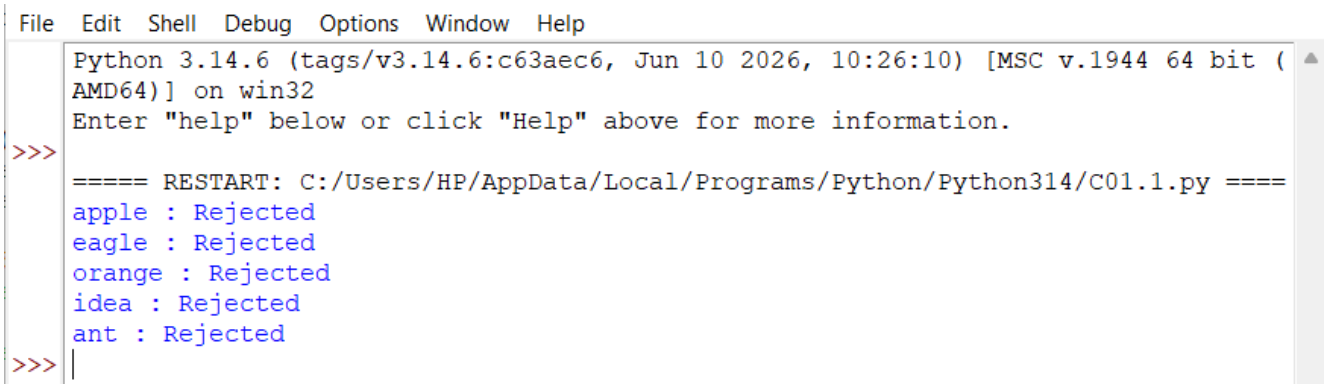
    for i in range(len(word) - 1):
        if word[i].lower() in vowels and word[i+1].lower() in vowels:
            consecutive_vowels = True

    if starts_vowel and ends_consonant and consecutive_vowels:
        return "Accepted"
    else:
        return "Rejected"


words = ["apple", "eagle", "orange", "idea", "ant"]

for word in words:
    print(word, ":", fsa(word))
```

OUTPUT:



Analysis and interpretation of results :

The FSA successfully checks the given words based on vowel and consonant patterns. It identifies words starting with vowels, ending with consonants, and containing consecutive vowels. The state transitions determine whether each word satisfies the required conditions. Valid and invalid words are classified as accepted or rejected.

BTL

Question 3: Morphological Analysis of English Words 4 – Analyze | 10 Marks Develop a Python program to perform morphological analysis on a given set of English words.

1. Identify the root word and its affixes.
2. Separate prefixes and suffixes wherever applicable.
3. Identify the morphological structure of each word.
4. Analyze words such as unhappiness, replaying, disagreement, and careless.
5. Display the morphological components in a structured format.

CODE:

```
prefixes = ["un", "re", "dis", "pre", "mis"]
suffixes = ["ness", "ing", "ment", "less", "ed"]

words = ["unhappiness", "replaying", "disagreement", "careless"]

for word in words:
    root = word
    prefix = ""
    suffix = ""

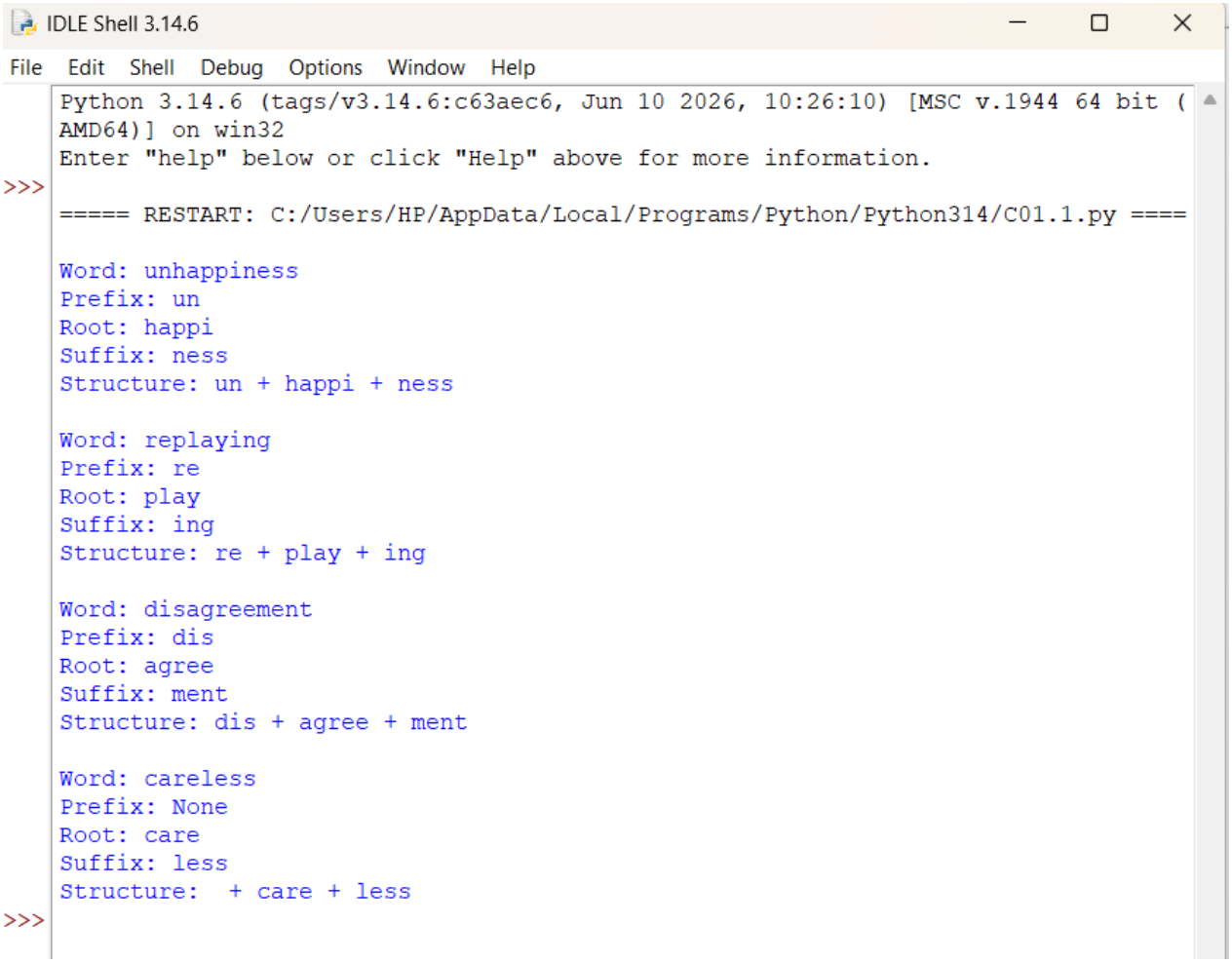
    for p in prefixes:
        if root.startswith(p):
            prefix = p
            root = root[len(p):]
            break

    for s in suffixes:
        if root.endswith(s):
            suffix = s
            root = root[:-len(s)]
            break

    print("\nWord:", word)
    print("Prefix:", prefix if prefix else "None")
    print("Root:", root)
    print("Suffix:", suffix if suffix else "None")
    print("Structure:", prefix + " + " + root + " + " + suffix)
```

OUTPUT:

BTL



Analysis and interpretation of results :

The program successfully identifies the root words, prefixes, and suffixes of the given English words. Words such as unhappiness, replaying, disagreement, and careless are divided into their morphological components. The results show how affixes change the meaning or grammatical form of root words.

Question 4: Inflectional and Derivational Morphology 3

– Apply | 10 Marks Write a Python program to identify and classify inflectional and derivational word forms from a given text.

1. Identify plural noun forms.
2. Identify past-tense and present participle verb forms.
3. Identify words formed using prefixes and suffixes.

BTL

4. Classify the identified words into inflectional and derivational categories.
5. Display the original word and its morphological classification.

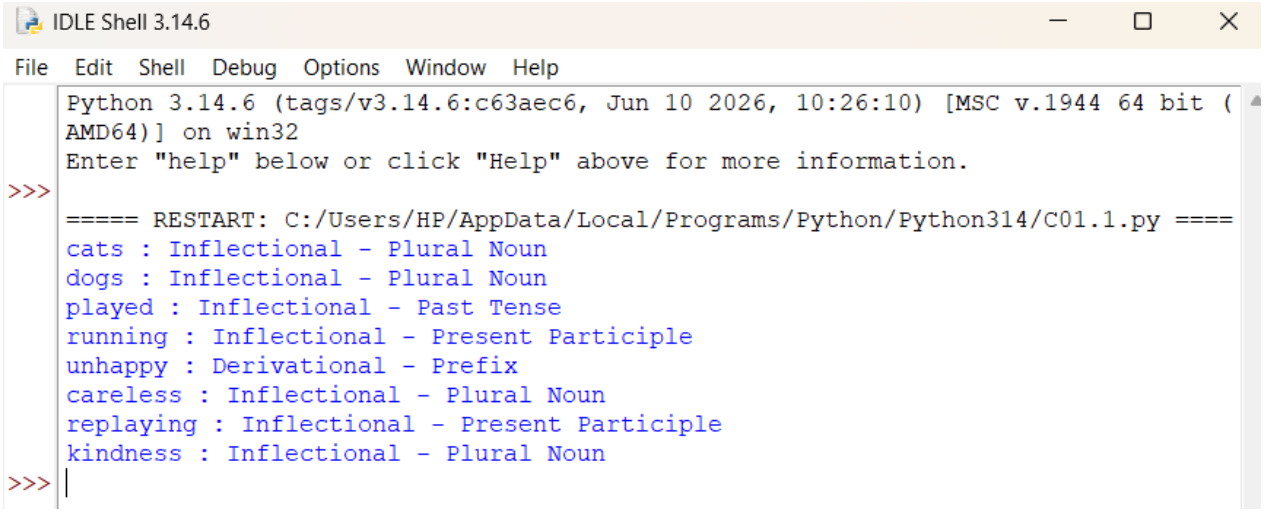
CODE:

```
words = [
    "cats", "dogs", "played", "running",
    "unhappy", "careless", "replaying", "kindness"
]

for word in words:
    if word.endswith(("s", "es")):
        category = "Inflectional - Plural Noun"
    elif word.endswith("ing"):
        category = "Inflectional - Present Participle"
    elif word.endswith("ed"):
        category = "Inflectional - Past Tense"
    elif word.startswith(("un", "re", "dis")):
        category = "Derivational - Prefix"
    elif word.endswith(("ness", "less", "ful")):
        category = "Derivational - Suffix"
    else:
        category = "Unknown"

    print(word, ":", category)
```

OUTPUT:



BTL

Analysis and interpretation of results :

The program identifies plural nouns, past-tense verbs, present participles, prefixes, and suffixes. It classifies words into inflectional and derivational categories based on their formation. The results show that inflectional morphology changes grammatical forms, whereas derivational morphology creates new words or changes word classes.

Question 5: Porter Stemming Algorithm 4 –

Analyze | 10 Marks Implement the Porter Stemming Algorithm using

Python to identify the root forms of English words.

1. Apply stemming to a given paragraph.
2. Display the original words and their stemmed forms.
3. Identify words that produce the same stem.
4. Compare the original and stemmed words.
5. Analyze how stemming reduces different word forms to a common base.

Example words: playing, played, studies, studying, connected, connection.

CODE:

```
from nltk.stem import PorterStemmer
from collections import defaultdict
import re

stemmer = PorterStemmer()

text = """
Playing played studies studying connected connection
running runs easily fairly
"""

words = re.findall(r'\b[a-zA-Z]+\b', text.lower())

groups = defaultdict(list)

print("Original Word\tStemmed Word")

for word in words:
    stem = stemmer.stem(word)
    print(word, "\t\t", stem)
    groups[stem].append(word)

print("\nWords with Same Stem:")

for stem, original in groups.items():
    if len(original) > 1:
        print(stem, ":", original)
```

OUTPUT:

BTL

```
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/HP/AppData/Local/Programs/Python/Python314/C01.1.py ====
Original Word   Stemmed Word
playing                 play
played          play
studies                 studi
studying                studi
connected               connect
connection              connect
running                 run
runs            run
easily          easili
fairly          fairli

Words with Same Stem:
play : ['playing', 'played']
studi : ['studies', 'studying']
connect : ['connected', 'connection']
run : ['running', 'runs']
>>>
```

Analysis and interpretation of results :

The Porter Stemming Algorithm successfully converts different word forms into their root stems. Words such as playing, played, and studying are reduced to shorter stem forms. The comparison shows that stemming reduces vocabulary variations and helps improve text-processing efficiency.

Question 6: Implementation of N-gram Language Models BTL 3 Apply | 10 Marks

Develop a Python program to implement unigram, bigram and trigram language models for a given English paragraph.

1. Tokenize the given paragraph into individual words.
2. Generate unigram, bigram and trigram sequences.
3. Calculate the frequency of each N-gram.
4. Calculate the probability of selected bigrams and trigrams.
5. Compare the generated N-gram models and display the results.

CODE:

```
from collections import Counter
import re

text = "I love natural language processing. I love Python programming."

words = re.findall(r'\b\w+\b', text.lower())

unigrams = words
bigrams = list(zip(words, words[1:]))
trigrams = list(zip(words, words[1:], words[2:]))

uni_count = Counter(unigrams)
bi_count = Counter(bigrams)
tri_count = Counter(trigrams)

print("Unigram Frequencies:")
print(uni_count)

print("\nBigram Frequencies:")
print(bi_count)

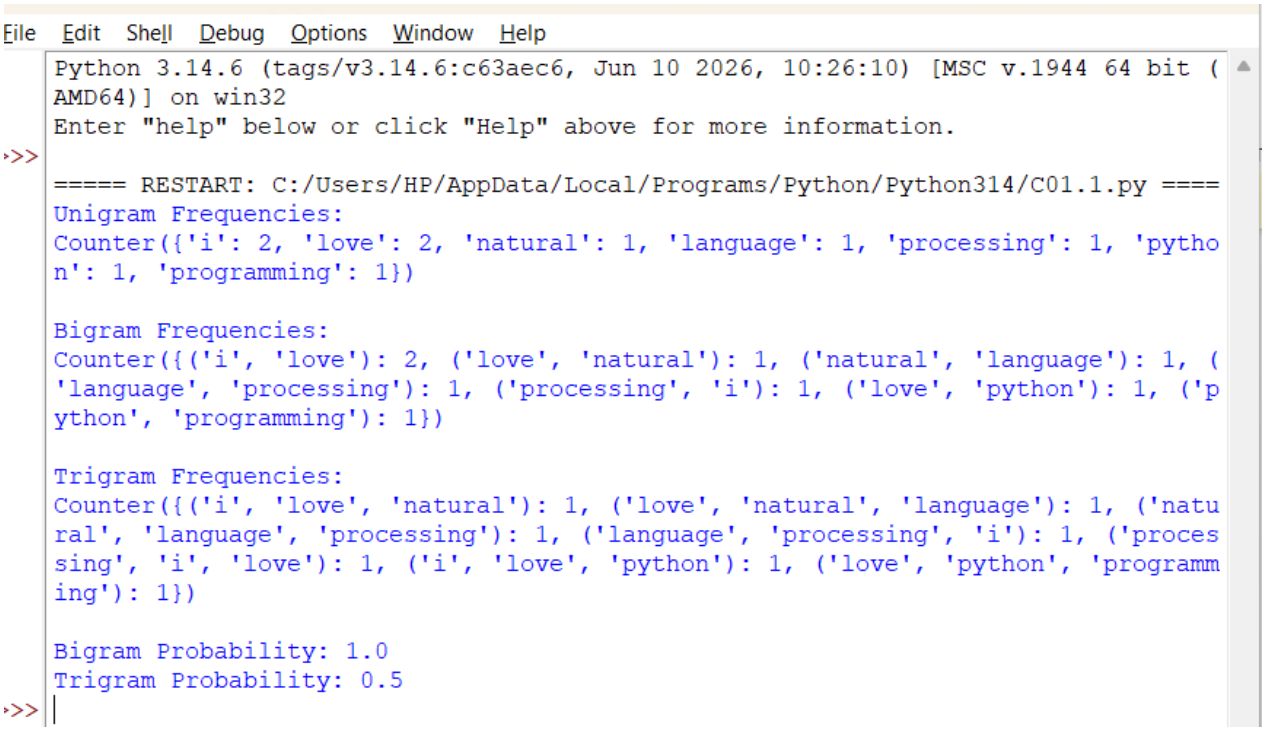
print("\nTrigram Frequencies:")
print(tri_count)

bigram = ("i", "love")
bigram_prob = bi_count[bigram] / uni_count[bigram[0]]

trigram = ("i", "love", "python")
trigram_prob = tri_count[trigram] / bi_count[trigram[:2]]

print("\nBigram Probability:", bigram_prob)
print("Trigram Probability:", trigram_prob)
```

OUTPUT:



Analysis and interpretation of results :

The program successfully generates unigram, bigram, and trigram sequences from the given paragraph. It calculates their frequencies and probabilities based on word occurrences. The results show that unigram models consider individual words, while bigram and trigram models capture word relationships and context.

Question 7: Word Frequency and N-gram Smoothing BTL 3 Apply | 10 Marks

Write a Python program to calculate word frequencies and implement smoothing techniques for an N-gram language model.

1. Count the occurrences of individual words in a given corpus.
2. Generate bigram frequencies.
3. Calculate unsmoothed bigram probabilities.
4. Implement Add-One (Laplace) smoothing to handle unseen bigrams.
5. Compare the probabilities obtained before and after smoothing.

CODE:

```
from collections import Counter
import re

text = "I love Python. I love programming."

words = re.findall(r'\b\w+\b', text.lower())

word_count = Counter(words)
bigrams = list(zip(words, words[1:]))
bigram_count = Counter(bigrams)

vocab_size = len(word_count)

print("Word Frequencies:")
print(word_count)

print("\nBigram Frequencies:")
print(bigram_count)

print("\nBigram Probabilities:")

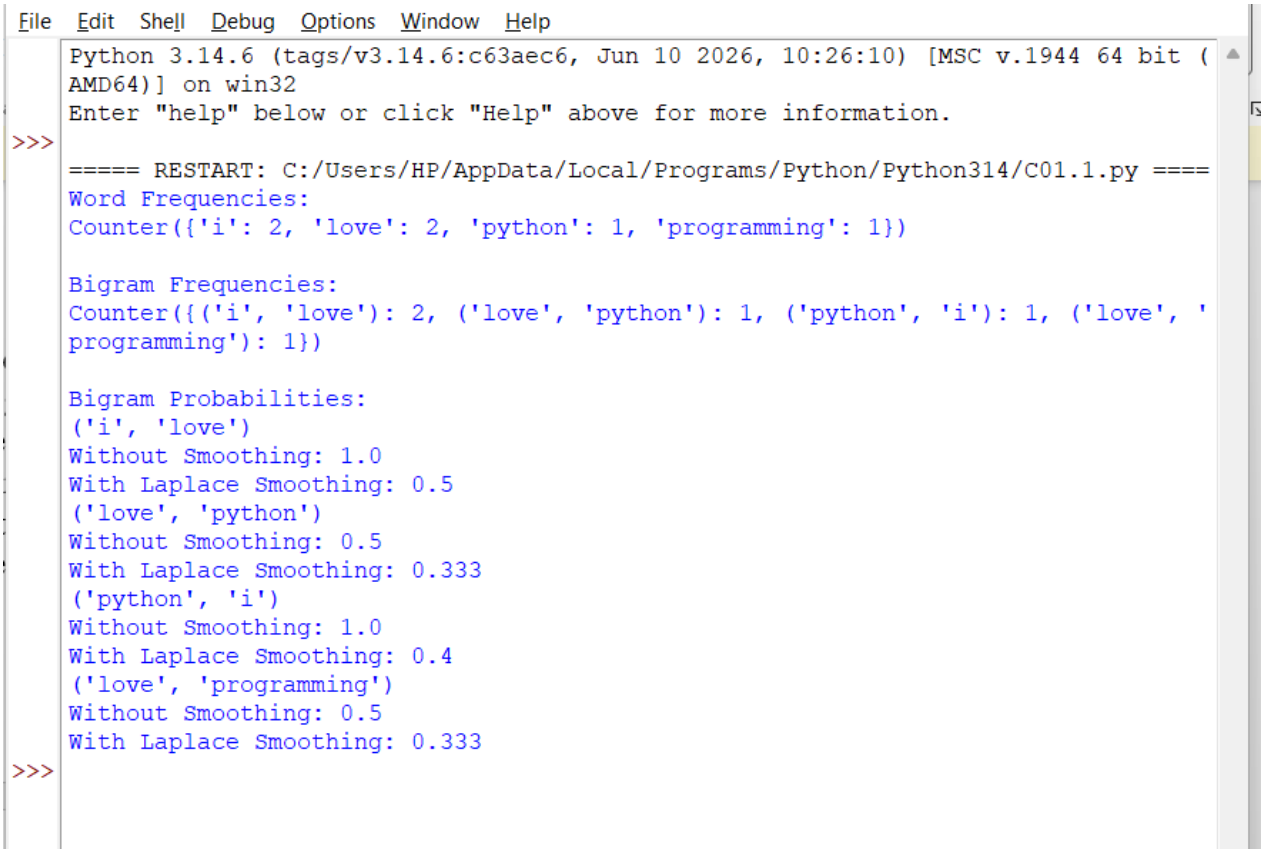
for bigram, count in bigram_count.items():
    first_word = bigram[0]

    unsmoothed = count / word_count[first_word]

    smoothed = (count + 1) / (
        word_count[first_word] + vocab_size
    )

    print(bigram)
    print("Without Smoothing:", round(unsmoothed, 3))
    print("With Laplace Smoothing:", round(smoothed, 3))
```

OUTPUT:



Analysis and interpretation of results :

The program successfully calculates word frequencies and bigram probabilities. Add-One (Laplace) smoothing assigns non-zero probabilities to unseen bigrams. The comparison shows that smoothing handles missing word combinations and improves the reliability of language models.

Question 8: Entropy Calculation for Language Models BTL 4

Analyze | 10 Marks Develop a Python program to calculate the entropy of a given text corpus using unigram and bigram probability distributions.

1. Tokenize the given corpus.
2. Calculate unigram and bigram probabilities.
3. Compute entropy using Shannon's entropy formula.
4. Compare the entropy values of unigram and bigram models.

5. Analyze how the context of preceding words influences the uncertainty of the next word.

CODE:

```
from collections import Counter
import math
import re

text = "I love Python. I love NLP. I like Python."

words = re.findall(r'\b\w+\b', text.lower())

unigrams = Counter(words)
bigrams = Counter(zip(words, words[1:]))

total_words = len(words)
total_bigrams = sum(bigrams.values())

# Unigram entropy
H1 = 0

for count in unigrams.values():
    p = count / total_words
    H1 -= p * math.log2(p)

# Conditional bigram entropy
H2 = 0

for (w1, w2), count in bigrams.items():
    p = count / unigrams[w1]
    joint_p = count / total_bigrams
    H2 -= joint_p * math.log2(p)

print("Unigram Entropy:", round(H1, 3))
print("Bigram Entropy:", round(H2, 3))
```

OUTPUT:

```
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/HP/AppData/Local/Programs/Python/Python314/C01.1.py ====
Unigram Entropy: 2.197
Bigram Entropy: 0.719
>>>
```

Analysis and interpretation of results :

The program successfully calculates unigram and bigram probabilities and their entropy values using Shannon's formula. The results show the uncertainty involved in predicting the next word. Bigram models consider the previous word as context, which can reduce prediction uncertainty compared with unigram models.

Assignment 9: English Part-of-Speech Tagging BTL 3 – Apply | 10 Marks Develop a Python program to perform Part-of-Speech (POS) tagging on a given English paragraph.

1. Tokenize the input paragraph into words.
2. Identify nouns, verbs, adjectives, adverbs and prepositions.
3. Apply an appropriate English POS tagset.
4. Display each word along with its corresponding POS tag.
5. Count and display the frequency of each POS category.

Suggested Python library: NLTK.

CODE:

```
import nltk
from collections import Counter

nltk.download('punkt')
nltk.download('averaged_perceptron_tagger_eng')

text = "The intelligent student quickly completed the Python assignment in college."

words = nltk.word_tokenize(text)

tagged_words = nltk.pos_tag(words)

print("Word\tPOS Tag")

categories = Counter()

for word, tag in tagged_words:
    print(word, "\t", tag)

    if tag.startswith("NN"):
        categories["Noun"] += 1
    elif tag.startswith("VB"):
        categories["Verb"] += 1
    elif tag.startswith("JJ"):
        categories["Adjective"] += 1
    elif tag.startswith("RB"):
        categories["Adverb"] += 1
    elif tag == "IN":
        categories["Preposition"] += 1

print("\nPOS Category Frequencies:")

for category, count in categories.items():
    print(category, ":", count)
```

OUTPUT:

```
Word     POS Tag
The      DT
intelligent      JJ
student          NN
quickly          RB
completed        VBD
the      DT
Python   NNP
assignment       NN
in       IN
college          NN
.        .

POS Category Frequencies:
Adjective : 1
Noun : 4
Adverb : 1
Verb : 1
Preposition : 1
>
```

Analysis and interpretation of results :

The NLTK POS tagger successfully tokenizes the paragraph and assigns grammatical tags to individual words. It identifies nouns, verbs, adjectives, adverbs, and prepositions. The results display each word with its POS tag and the frequency of each grammatical category.

Assignment 10: Comparative Analysis of POS Tagging Technique BTL 4 – Analyze | 10 Marks Develop a Python application to analyze and compare Rule-Based, Stochastic and Transformation-Based POS tagging techniques.

1. Implement a rule-based POS tagger using predefined grammatical rules.
2. Apply a stochastic POS tagger using a trained probabilistic model.
3. Implement or demonstrate transformation-based tagging using an available Python NLP library or a simplified transformation-rule approach.
4. Compare the tagging results for the same set of sentences.
5. Analyze the differences in tagging accuracy and identify cases where the tagging methods produce different results.Suggested Python libraries: NLTK and spaCy.

CODE:

```python
import nltk
import re
from nltk.tag import RegexpTagger, UnigramTagger

# Training Data
train_data = nltk.corpus.treebank.tagged_sents()

# 1. Rule-Based Tagger
rules = [
    (r'.*ing$', 'VBG'),
    (r'.*ed$', 'VBD'),
    (r'.*ly$', 'RB'),
    (r'.*s$', 'NNS'),
    (r'.*', 'NN')
]

rule_tagger = RegexpTagger(rules)

# 2. Stochastic Tagger
stochastic_tagger = UnigramTagger(train_data)

# 3. Transformation-Based Tagger
def transformation_tagger(words):
    tagged = rule_tagger.tag(words)

    for i, (word, tag) in enumerate(tagged):
        w = word.lower()

        if w in ['the', 'a', 'an']:
            tagged[i] = (word, 'DT')

        elif w in ['is', 'am', 'are', 'was', 'were']:
            tagged[i] = (word, 'VBZ' if w == 'is' else 'VBP')

        elif w in ['in', 'on', 'at', 'by', 'with', 'of']:
            tagged[i] = (word, 'IN')

        elif word in ['.', ',', '?', '!']:
            tagged[i] = (word, '.')

    return tagged
```

```python
# Input Sentences
sentences = [
    "The students are playing happily in the garden.",
    "The intelligent boy completed his homework.",
    "She is reading a beautiful book."
]

# Compare Three Techniques
for sentence in sentences:

    words = re.findall(r'\w+|[^\w\s]', sentence)

    print("\nSentence:", sentence)

    print("\n1. Rule-Based Tagging:")
    print(rule_tagger.tag(words))

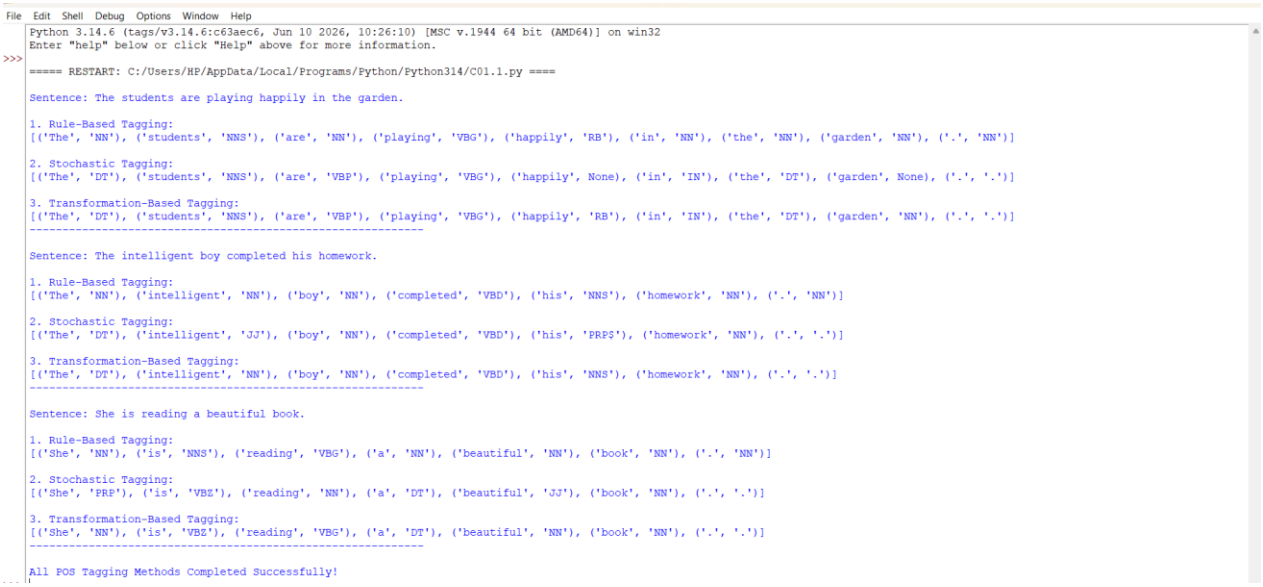
    print("\n2. Stochastic Tagging:")
    print(stochastic_tagger.tag(words))

    print("\n3. Transformation-Based Tagging:")
    print(transformation_tagger(words))

    print("-" * 60)

print("\nAll POS Tagging Methods Completed Successfully!")
```

OUTPUT:



```
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/HP/AppData/Local/Programs/Python/Python314/C01.1.py ====

Sentence: The students are playing happily in the garden.

1. Rule-Based Tagging:
[('The', 'NN'), ('students', 'NNS'), ('are', 'NN'), ('playing', 'VBG'), ('happily', 'RB'), ('in', 'NN'), ('the', 'NN'), ('garden', 'NN'), ('.', 'NN')]

2. Stochastic Tagging:
[('The', 'DT'), ('students', 'NNS'), ('are', 'VBP'), ('playing', 'VBG'), ('happily', None), ('in', 'IN'), ('the', 'DT'), ('garden', None), ('.', '.')]

3. Transformation-Based Tagging:
[('The', 'DT'), ('students', 'NNS'), ('are', 'VBP'), ('playing', 'VBG'), ('happily', 'RB'), ('in', 'IN'), ('the', 'DT'), ('garden', 'NN'), ('.', '.')]
------------------------------------------------------------

Sentence: The intelligent boy completed his homework.

1. Rule-Based Tagging:
[('The', 'NN'), ('intelligent', 'NN'), ('boy', 'NN'), ('completed', 'VBD'), ('his', 'NNS'), ('homework', 'NN'), ('.', 'NN')]

2. Stochastic Tagging:
[('The', 'DT'), ('intelligent', 'JJ'), ('boy', 'NN'), ('completed', 'VBD'), ('his', 'PRP$'), ('homework', 'NN'), ('.', '.')]

3. Transformation-Based Tagging:
[('The', 'DT'), ('intelligent', 'NN'), ('boy', 'NN'), ('completed', 'VBD'), ('his', 'NNS'), ('homework', 'NN'), ('.', '.')]
------------------------------------------------------------

Sentence: She is reading a beautiful book.

1. Rule-Based Tagging:
[('She', 'NN'), ('is', 'NNS'), ('reading', 'VBG'), ('a', 'NN'), ('beautiful', 'NN'), ('book', 'NN'), ('.', 'NN')]

2. Stochastic Tagging:
[('She', 'PRP'), ('is', 'VBZ'), ('reading', 'NN'), ('a', 'DT'), ('beautiful', 'JJ'), ('book', 'NN'), ('.', '.')]

3. Transformation-Based Tagging:
[('She', 'NN'), ('is', 'VBZ'), ('reading', 'VBG'), ('a', 'DT'), ('beautiful', 'NN'), ('book', 'NN'), ('.', '.')]
------------------------------------------------------------

All POS Tagging Methods Completed Successfully!
```

Analysis and interpretation of results :

The program successfully compares rule-based, stochastic, and transformation-based POS tagging techniques. Each method assigns grammatical tags using different approaches, such as predefined rules, probabilities, and transformation rules. The comparison shows that contextual information improves tagging performance and helps identify differences between the techniques.

Question	Major Concept	BTL Level	Marks
1	Regular Expressions	3	10
2	Finite State Automata	3	10
3	Morphological Analysis	4	10
4	Inflectional and Derivational Morphology	3	10
5	Porter Stemmer	4	10
6	N-gram Models	3	10
7	Word Counting and Smoothing	3	10
8	Entropy	4	10
9	POS Tagging	3	10
10	POS Tagging Techniques	4	10

Evaluation Rubric (10 Marks per Assignment)

Evaluation Criteria	**Marks**
Understanding of the NLP concept	2
Python implementation and logic	3
Correctness of output	2
Analysis and interpretation of results	2
Code readability and documentation	1
Total	10